



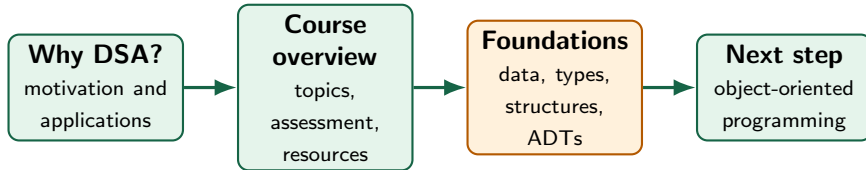
Data Structures and Algorithms in Python

Lecture 1 – Part II: Programming Foundations and ADTs

Faculty of Data Science and Artificial Intelligence

College of Technology - National Economics University

Where we are in Lecture 1



Main question for this half

Before designing efficient algorithms, how should we represent, organize, and interact with data?



Learning outcomes

By the end of this part, you should be able to:

- 1 explain how Python represents data as values accessed through variable names;
- 2 distinguish primitive/basic values from collection types;
- 3 define, classify, and choose a data structure from the operations a problem requires;
- 4 explain an Abstract Data Type (ADT) as a public contract;
- 5 distinguish an ADT from its concrete implementation.

Data → Values → Types → Organization → Operations



SECTION 1

Data

Data in a Python program

What is data?

Data are facts or observations that a program stores, processes, and produces.

How Python represents data

Python represents data as **values**. A variable gives a value a meaningful name so the program can use it.

```
1 student_name = "An"  
2 student_id = 2026001  
3 gpa = 3.42  
4 is_active = True
```

Examples of represented data

Text, identifiers, measurements, and logical conditions become Python values that a program can manipulate.



Working with data through variables

```
1 balance = 1000
2 deposit = 250
3
4 balance = balance + deposit
5 print(balance) # 1250
```

Names make data usable

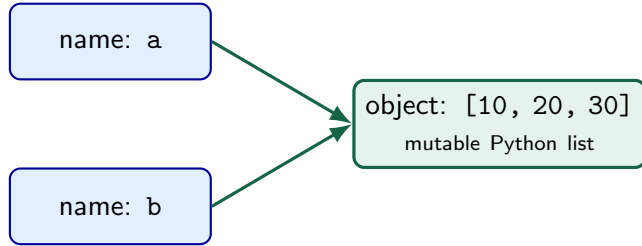
`balance` and `deposit` describe the meaning of their values and allow those values to be used in an expression.

Assignment updates program state

The new result is assigned to `balance`.
The program can then read the updated value later.



The same mutable data can have two names



```
1 a = [10, 20, 30]
2 b = a
3 b.append(40)
4 print(a) # [10, 20, 30, 40]
```

Why does a change?

Both names refer to the same mutable object.



Quick check: shared data or copied data?

Predict both outputs

```
1 scores = [8, 7]
2 shared = scores
3 copied = scores.copy()
4 shared.append(9)
5 print(scores)
6 print(copied)
```



Quick check: shared data or copied data?

Predict both outputs

```
1 scores = [8, 7]
2 shared = scores
3 copied = scores.copy()
4 shared.append(9)
5 print(scores)
6 print(copied)
```

Answer

scores: [8, 7, 9]

copied: [8, 7]

Why?

shared refers to the same list as scores, whereas copy() creates an independent list.

Choose deliberately between sharing mutable data and copying it.



SECTION 2

Data Types

What is a data type?

Definition

A data type identifies a set of possible values and the operations that are meaningful for those values.

Why types matter

Types help the language interpret stored bits and decide which operations are valid.

int

whole numbers

float

real-valued
approximations

str

text sequences

bool

True / False



Primitive (basic) data types in Python

Type	Example	Typical meaning	Example operation
int	2026001	identifier, count	+, -, //
float	3.42	measurement, score	+, *, /
str	"Nguyen An"	text	concatenation, indexing
bool	True	logical condition	and, or, not

Course convention

We use *primitive/basic* for these fundamental built-in values. Python treats every value as an object and has no separate char type; "A" is a string of length one.



The same operator may behave differently

```
1 print(2 + 3) # 5
2 print(2.0 + 3.0) # 5.0
3 print("data" + "science") # datascience
4 print([1, 2] + [3, 4]) # [1, 2, 3, 4]
5
6 print(2 + "3") # TypeError
```

No automatic conversion

Python does not silently convert "3" to the integer 3. Use `2 + int("3")` when conversion is intended.

Key lesson

A data type determines which operators are valid, what they mean, and the type of result they produce.

Type-directed meaning of +

- `int + int` → integer addition;
- `float + float` → floating-point addition;
- `str + str` → string concatenation;
- `list + list` → list concatenation.

Runtime decision

Python examines the operand types and selects the operation supported by those types. An unsupported combination raises `TypeError`.



Non-primitive built-in collection types

Sequences

- `list`: ordered and mutable
- `tuple`: ordered and immutable

Examples

```
[500000, -200000, 1000000]  
("Hanoi", 21.5)
```

Mappings and sets

- `dict`: key-value mapping
- `set`: unique values

Examples

```
{"id": 105, "balance": 2.5e7}  
{"ATM", "Mobile Banking"}
```



Primitive versus non-primitive data types

Aspect	Primitive / basic	Non-primitive / composite
Main role	Represent a fundamental value directly	Organize multiple values or fields
Python examples	int, float, bool, str	list, tuple, dict, set, user-defined classes
Typical operations	Arithmetic, comparison, logic, and text operations	Indexing, iteration, membership, key lookup, and object methods
Mutability	The examples above are immutable	May be mutable or immutable



Choosing a collection by behavior

Requirement	Order?	Duplicates?	Candidate
Daily transaction sequence	Yes	Yes	list
Geographic coordinate	Yes	Yes	tuple
Customer record by field name	By key	Keys unique	dict
Unique registered services	No fixed order	No	set

Choose a type because of the behavior the problem requires.



SECTION 3

Data Structures

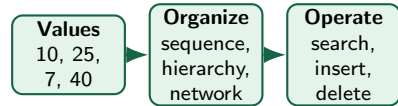
From data types to data structures

Data type asks

What kind of value is this, and which operations are valid?

Data structure asks

How should many values be organized so that important operations can be performed efficiently?



Definition

A data structure is a particular way of storing and organizing data so it can be used effectively.



Linear and non-linear data structures

Linear structures

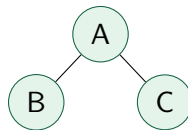
Elements are logically arranged in a sequence.



Python lists, arrays, and linked lists

Non-linear structures

Elements form hierarchical or network relationships.



Trees and graphs



Example: choose a structure for the workload

Scenario and required operations

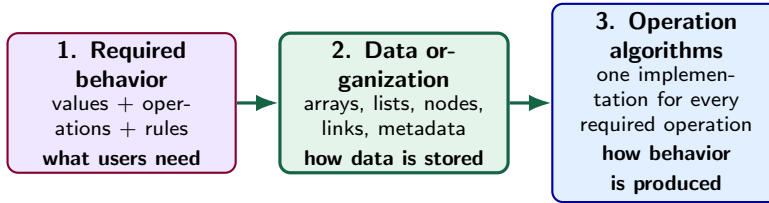
Store the unique student IDs allowed to enter a laboratory. `contains(id)` is very frequent; `add(id)` and `remove(id)` are occasional; order and index positions are not needed.

Candidate	Membership test	Add / remove	Other behavior
List	May require scanning items	Append is easy; removal may shift items	Preserves order and indexes; duplicates require checking
Set	Fast on average	Fast on average	Enforces unique values; no index positions
Dictionary	Fast key lookup on average	Fast by key on average	Best when each ID must map to an associated record

Decision: choose a set. It directly optimizes membership, updates, and uniqueness. Use a dictionary only when each ID maps to a record.



Designing a data structure: three required parts



Design part	Question to answer	Python list example
Public contract	Which values, operations, results, and rules are visible?	Ordered mutable sequence: access, update, append, insert, and remove
Data organization	How are elements and supporting state arranged?	Elements are maintained in index order by a built-in list object
Operation implementation	Which algorithm realizes each operation and preserves the rules?	<code>items[i]</code> , <code>items[i] = x</code> , <code>append</code> , <code>insert</code> , and <code>pop</code>

Design order: specify required behavior → organize the data → implement and test every operation.



SECTION 4

Abstract Data Types

Abstract Data Type (ADT)

Definition

An ADT specifies a collection of data together with the operations and behavioral rules available to users.

Focus

The ADT describes **what** can be done, not exactly **how** it is implemented.

ADT specification

Data model

values and state

Operations

commands
and queries

Behavioral rules

valid outcomes
and constraints



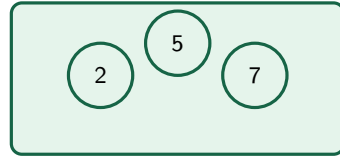
Example ADT: Set

Data

A **Set ADT** represents a collection of **unique elements**.

Operations

- `add(x)`, `remove(x)`
- `contains(x)`
- `size()`, `is_empty()`
- `union(S)`
- `intersection(S)`
- `difference(S)`



$$S = \{2, 5, 7\}$$

Rules

- No duplicate values.
- No index positions.
- Adding an existing element changes nothing.



Set ADT: what vs. how



User knows

- Supported operations
- Duplicate rule
- Returned results

User need not know

- Internal storage
- Membership checking
- Duplicate prevention

$S = \{2, 5, 7\}$, $\text{add}(5) \Rightarrow S = \{2, 5, 7\}$

ADT specifies behavior; implementation decides how.



ADT versus data structure

Abstract Data Type	Data Structure / Implementation
Describes required behavior	Describes concrete organization
Focuses on operations and rules	Focuses on memory and algorithms
Answers: "What can the user do?"	Answers: "How is it built?"
Examples: Set, sequence, key-value map	Examples: array, linked list, hash table

ADT = specification

Data structure = realization



Activity: unique course registrations

Scenario

A registration program stores course codes. A course may appear only once, and the program must add, remove, and check registered courses.

Questions

- 1 What data should be stored?
- 2 What is the type of one item?
- 3 Which ADT describes the required behavior?
- 4 How can a Python `list` implement it?



Activity: unique course registrations

Scenario

A registration program stores course codes. A course may appear only once, and the program must add, remove, and check registered courses.

Questions

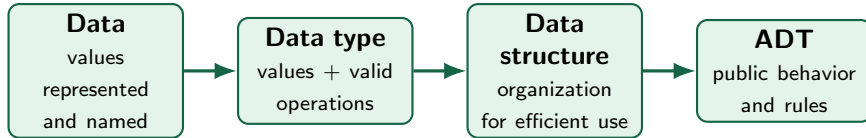
- 1 What data should be stored?
- 2 What is the type of one item?
- 3 Which ADT describes the required behavior?
- 4 How can a Python list implement it?

Suggested answer

- data: registered course codes;
- item type: `str`;
- ADT: Set, because duplicates are forbidden;
- implementation: use `code` in `courses`, append only when absent, and use `courses.remove(code)`.



Connecting the four concepts



Central lesson

Good program design separates what users need from how data is stored and manipulated internally.



Exit ticket

- 1 In `scores = [8, 7, 9]`, identify the data, variable name, and object type.
- 2 Why can the same Set ADT be implemented with either a Python list or a built-in set?
- 3 Classify each as an ADT or a concrete implementation: Set ADT, Python list, Python set, linked list.

Prepare for the next topic

Next, object-oriented programming will package related state, operations, and rules inside classes that can implement reusable ADTs.



Key takeaway

Programs represent data as values accessed through variable names. Types define valid values and operations. Data structures organize collections of values. ADTs define the public behavior independently of implementation. These distinctions allow us to design software that is clearer, safer, and easier to optimize.

Next: object-oriented programming.



Any Questions?

